LAPORAN PRAKTIKUM
WEB DESAIN
MODUL 2

Dosen Pengampu: Dede Irawan, M.Kom

Disusun oleh :

Nama : Chalista Erika Putri

NIM : 20250910058

Kelas : SINFC-02-2025

FAKULTAS ILMU KOMPUTER
UNIVERSITAS KUNINGAN
2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah SWT karena atas rahmat dan karunia-Nya, laporan praktikum Mata Kuliah Web Desain untuk Modul 2 yang membahas mengenai Git dan GitHub ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan kegiatan praktikum yang bertujuan untuk memahami konsep *Version Control System* (VCS), instalasi dan konfigurasi Git, perintah dasar pengelolaan proyek, serta kerja kolaboratif menggunakan *remote repository* GitHub. Ucapan terima kasih penulis sampaikan kepada Bapak Dede Irawan, M.Kom selaku dosen pengampu mata kuliah Web Desain, serta para asisten praktikum yang telah memberikan bimbingan selama proses praktikum berlangsung.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan. Oleh karena itu, saran dan kritik yang membangun sangat diharapkan demi penyempurnaan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi penulis maupun pembaca.

Kuningan, Oktober 2026
Penulis,

Chalista Erika Putri

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
DAFTAR GAMBAR iii
BAB I PRE-TEST 1
BAB II PEMBAHASAN PRAKTIKUM 2
A. Pembuatan Akun GitHub dan Konfigurasi Git 2
B. Mengelola Proyek Lokal dan Sinkronisasi ke GitHub 3
C. Hasil Sinkronisasi pada GitHub 4
BAB III POST-TEST 5
BAB IV TUGAS / LATIHAN 6
1. Alur Pembuatan Repository, Commit, dan Push 6
2. Rancangan Workflow Git Menggunakan Branch 6
3. Menyimpan Praktikum Modul 1 ke Repository Git 7
BAB V KESIMPULAN 8
DAFTAR PUSTAKA 8

DAFTAR GAMBAR

Gambar 2.1 Alur kerja perubahan berkas pada Git 2
Gambar 2.2 Tampilan repository laprak-webdesain-modul1 pada GitHub 4
Gambar 4.1 Alur kerja Git menggunakan branch fitur 7

BAB I
PRE-TEST

1. **Apa tujuan dari menggunakan *version control* dalam pengembangan perangkat lunak?**

   Tujuan utama penggunaan *Version Control System* (VCS) adalah untuk melacak dan mendokumentasikan setiap riwayat perubahan kode proyek secara terorganisir. Dengan VCS, pengembang dapat:

   - mengembalikan proyek ke versi sebelumnya (*rollback*) apabila terjadi kesalahan;
   - berkolaborasi dalam tim tanpa saling menimpa pekerjaan satu sama lain; dan
   - menyimpan cadangan riwayat pengkodean secara aman.

2. **Jelaskan perbedaan antara `git add` dan `git commit`!**

   Kedua perintah merupakan tahapan yang berurutan dalam menyimpan perubahan. Perbedaannya dirangkum pada Tabel 1.1.

   Tabel 1.1 Perbedaan `git add` dan `git commit`

Aspek	`git add`	`git commit`
Fungsi	Menandai berkas yang siap untuk disimpan	Merekam dan menyimpan perubahan secara permanen beserta pesan perubahan (*commit message*)
Perpindahan	*Working directory → staging area*	*Staging area → local repository*
Contoh	`git add index.html`	`git commit -m "Tambah halaman"`

3. **Bagaimana cara membuat *branch* baru di Git?**

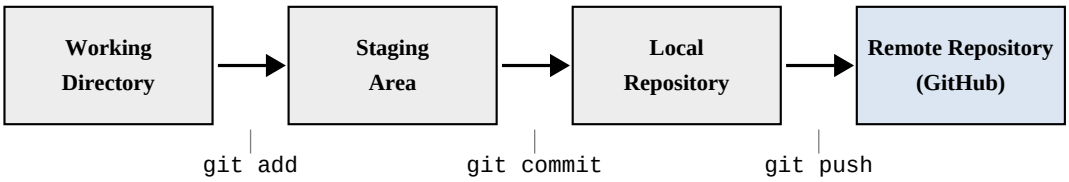
   Branch baru dapat dibuat dengan salah satu perintah berikut.

```
git branch <nama_branch>        # buat branch, tanpa berpindah
git checkout -b <nama_branch>   # buat branch dan berpindah
git switch -c <nama_branch>     # alternatif checkout -b
```

BAB II
PEMBAHASAN PRAKTIKUM

Praktikum Modul 2 membahas penggunaan Git sebagai *Version Control System* dan GitHub sebagai layanan *remote repository*. Tujuan praktikum ini adalah memahami konsep VCS, menginstal dan mengonfigurasi Git, menggunakan perintah dasar pengelolaan proyek, serta menyinkronkan *repository* lokal ke GitHub. Perangkat yang digunakan meliputi sistem operasi Windows, Git, Visual Studio Code, peramban web, dan akun GitHub.

Secara umum, setiap perubahan berkas pada Git melewati empat area seperti ditunjukkan pada Gambar 2.1: *working directory* (tempat berkas dikerjakan), *staging area* (tempat berkas disiapkan untuk disimpan), *local repository* (riwayat versi pada komputer), dan *remote repository* (riwayat versi pada GitHub).



Gambar 2.1 Alur kerja perubahan berkas pada Git

A. Pembuatan Akun GitHub dan Konfigurasi Git

1. Mendaftar akun resmi di GitHub melalui situs `https://github.com` dengan mengisikan *email*, *password*, dan *username*, kemudian menyelesaikan verifikasi keamanan hingga berhasil *login*.
2. Mengunduh dan menginstal aplikasi Git pada sistem operasi Windows. Keberhasilan instalasi diperiksa dengan perintah `git --version`.
3. Membuka terminal Visual Studio Code (`Ctrl + ``), lalu melakukan konfigurasi identitas global menggunakan perintah berikut.

```
git config --global user.name "Chalista Erika Putri"
git config --global user.email "email@domain.com"
```

Identitas tersebut akan tercatat pada setiap *commit* yang dibuat. Hasil konfigurasi dapat diperiksa dengan perintah `git config --list`.

B. Mengelola Proyek Lokal dan Sinkronisasi ke GitHub

1. Inisialisasi *repository*. Folder kerja praktikum dibuka pada VS Code, kemudian Git diinisialisasi menggunakan perintah berikut. Perintah ini membuat folder tersembunyi `.git` yang menyimpan seluruh riwayat versi.

```
git init
```

2. *Staging* dan *commit*. Berkas Latihan Modul 1 dipindahkan ke *staging area* menggunakan `git add`, lalu dilakukan *commit* pertama. Perintah `git status` digunakan untuk memeriksa status berkas sebelum dan sesudah proses tersebut.

```
git status
git add Latihan_01_01.html
git commit -m "Initial commit: Latihan_01_01.html"
```

3. Koneksi *remote* dan *push*. Langkah-langkahnya adalah sebagai berikut.

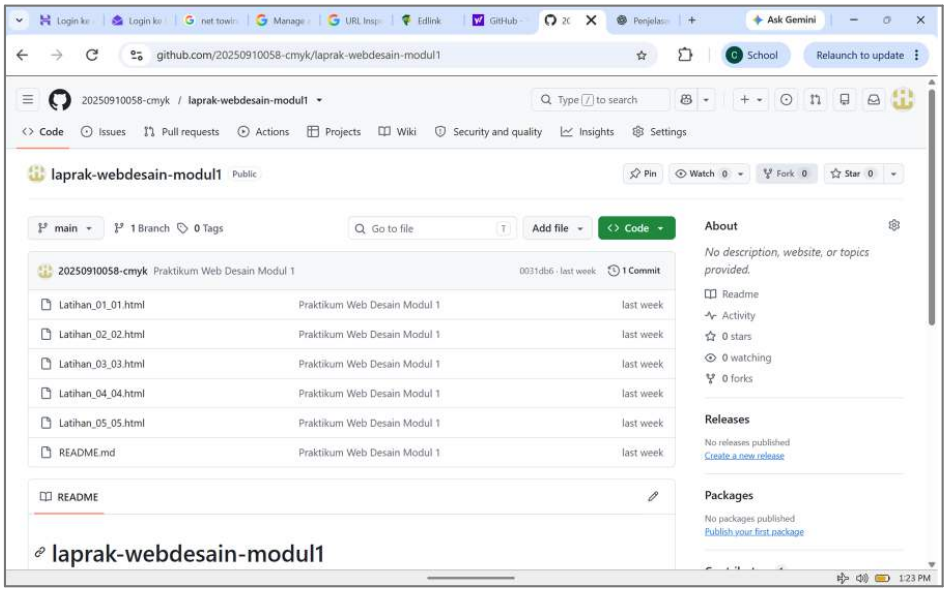
1. Membuat *repository* baru bernama `laprak-webdesain-modul1` pada akun GitHub.
2. Menghubungkan *repository* lokal ke *remote server* dengan URL `https://github.com/20250910058-cmyk/laprak-webdesain-modul1.git`, mengubah nama cabang utama menjadi `main`, lalu mengunggah seluruh riwayat perubahan ke GitHub.

```
git remote add origin <URL_repository>
git branch -M main
git push -u origin main
```

Opsi `-u` menetapkan *origin main* sebagai tujuan *push* bawaan, sehingga pengiriman berikutnya cukup menggunakan perintah `git push`.

C. Hasil Sinkronisasi pada GitHub

Setelah proses *push* berhasil, berkas praktikum tampil pada halaman *repository* GitHub seperti pada Gambar 2.2. *Repository* berstatus *Public* dengan satu *branch* (`main`) dan satu *commit*, serta memuat berkas `Latihan_01_01.html` sampai `Latihan_05_05.html` dan `README.md`.



Gambar 2.2 Tampilan *repository* laprak-webdesain-modul1 pada GitHub

BAB III
POST-TEST

1. **Apa fungsi dari `git merge` dalam *workflow* Git?**

   Fungsi `git merge` adalah menggabungkan riwayat *commit* dan perubahan kode dari satu *branch* (misalnya *branch* fitur atau eksperimen) ke *branch* tujuan (misalnya *branch* utama `main`). Penggabungan dilakukan dari *branch* tujuan, yaitu `git checkout main` lalu `git merge <nama_branch>`.

2. **Jelaskan cara mengatasi konflik (*merge conflict*) saat melakukan *merge*!**

   Konflik terjadi ketika dua *branch* mengubah baris yang sama sehingga Git tidak dapat menggabungkannya secara otomatis. Langkah penyelesaiannya:

   1. Membuka berkas yang mengalami konflik pada penyunting teks. VS Code menandai bagian konflik dengan `<<<<<<<`, `=======`, dan `>>>>>>>` seperti contoh berikut.

```
<<<<<<< HEAD
<h1>Judul dari branch main</h1>
=======
<h1>Judul dari branch fitur</h1>
>>>>>>> feature/tampilan-css
```

   2. Memilih baris kode yang dipertahankan (*Accept Current Change*, *Accept Incoming Change*, atau gabungan keduanya), lalu menghapus seluruh penanda konflik.
   3. Menyimpan berkas, memasukkannya kembali ke *staging area* dengan `git add <nama_file>`, kemudian menyelesaikan *merge* dengan `git commit -m "Fix merge conflict"`. Penggabungan dapat dibatalkan dengan `git merge --abort`.

3. **Tulis langkah-langkah untuk menambahkan file baru ke *repository* Git dan melakukan *commit*!**

   1. Membuat atau mengubah berkas baru di dalam direktori proyek.
   2. Memeriksa daftar berkas yang belum dilacak (*untracked*) dengan `git status`.
   3. Memasukkan berkas ke *staging area* dengan `git add <nama_file>` (atau `git add .` untuk seluruh berkas).
   4. Menyimpan perubahan dengan `git commit -m "Pesan perubahan yang jelas"`.

BAB IV
TUGAS / LATIHAN

1. Alur Pembuatan Repository, Commit, dan Push ke GitHub

Alur pembuatan *repository* lokal hingga terunggah ke GitHub ditunjukkan pada Tabel 4.1.

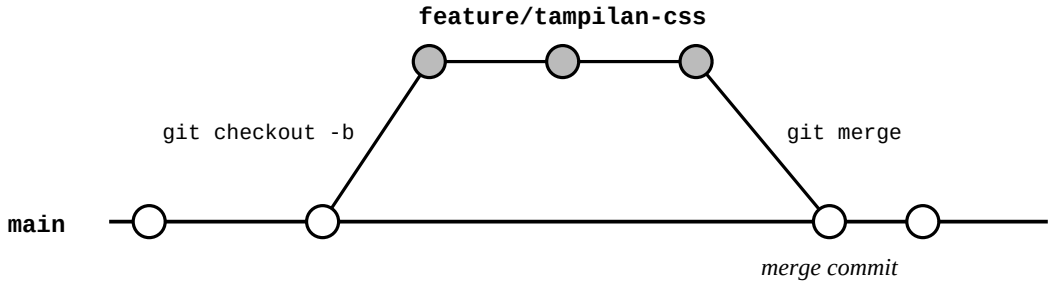
Tabel 4.1 Alur pembuatan *repository*, *commit*, dan *push*

No	Perintah	Keterangan
1	`git init`	Menginisialisasi *repository* lokal pada direktori proyek
2	`git add .`	Menambahkan seluruh berkas proyek ke *staging area*
3	`git commit -m "Add project files"`	Menyimpan versi awal proyek
4	`git branch -M main`	Menetapkan nama cabang utama menjadi `main`
5	`git remote add origin <URL_Repository>`	Menghubungkan direktori lokal ke *remote* GitHub
6	`git push -u origin main`	Mengunggah kode ke GitHub

2. Rancangan Workflow Git Menggunakan Branch

Rancangan *workflow* menggunakan tiga komponen berikut, yang diilustrasikan pada Gambar 4.1.

- `main`: cabang utama yang berisi kode *production-ready* (stabil dan siap digunakan).
- `feature/<nama-fitur>`: cabang terpisah yang dibuat dari `main` khusus untuk mengembangkan fitur baru, misalnya `feature/tampilan-css`.
- Integrasi (*merge*): setelah fitur selesai dibuat dan diuji, pengembang kembali ke `main` lalu menggabungkan kode dari cabang fitur tersebut.



Gambar 4.1 Alur kerja Git menggunakan *branch* fitur

Perintah yang digunakan pada alur tersebut adalah sebagai berikut.

```
git checkout -b feature/tampilan-css
git add .
git commit -m "Menambahkan tampilan CSS"
git checkout main
git merge feature/tampilan-css
git branch -d feature/tampilan-css
git push origin main
```

Perintah `git branch -d` menghapus cabang fitur yang sudah digabungkan sehingga daftar *branch* tetap rapi.

3. Menyimpan Praktikum Modul 1 ke Repository Git

Seluruh berkas praktikum Modul 1 disimpan ke *repository* Git dengan langkah pada Tabel 4.2. Hasilnya dapat dilihat pada Gambar 2.2.

Tabel 4.2 Langkah penyimpanan praktikum Modul 1 ke *repository*

No	Perintah	Keterangan
1	–	Menyalin seluruh berkas praktikum Modul 1 ke dalam folder proyek Git
2	`git add .`	Memasukkan seluruh berkas ke *staging area*
3	`git commit -m "Praktikum Web Desain Modul 1"`	Menyimpan perubahan ke *local repository*
4	`git push origin main`	Mengirim perubahan ke *repository* GitHub

BAB V
KESIMPULAN

Berdasarkan praktikum Modul 2 yang telah dilaksanakan, dapat disimpulkan bahwa:

1. Git merupakan sistem kontrol versi (*Version Control System*) berarsitektur terdistribusi yang efektif untuk memantau setiap perubahan berkas pada proyek pengembangan perangkat lunak.
2. GitHub sebagai *remote repository* memungkinkan penyimpanan kode berbasis *cloud* yang aman serta memfasilitasi kerja kolaboratif antarpengembang secara terstruktur.
3. Perintah dasar seperti `git init`, `add`, `commit`, `branch`, `merge`, dan `push` merupakan fondasi utama dalam mengelola kode proyek secara baik dan profesional.
4. Penggunaan *branch* memungkinkan pengembangan fitur dilakukan secara terpisah dari kode utama, sehingga cabang `main` tetap stabil.

DAFTAR PUSTAKA

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book

Git. (t.t.). *Git Documentation*. https://git-scm.com/docs

GitHub. (t.t.). *GitHub Docs*. https://docs.github.com